

目录

1	目前已经（部分）实现的函数列表	2
1.1	样条函数生成	2
1.2	样条函数后置处理	2
1.3	断点、节点、位点	2
2	函数功能说明与使用样例	2
2.1	样条函数生成	2
2.1.1	csapi	2
2.1.2	csape	3
2.1.3	ppmak	4
2.1.4	bspline	6
2.1.5	spapi	6
2.1.6	spmak	8
2.2	样条函数后置处理	9
2.2.1	fnval	9
2.2.2	fn2fm	9
2.2.3	fnder	9
2.2.4	fnbrk	11
2.3	断点、节点、位点	12
2.3.1	aptknt	12
2.3.2	augknt	13
2.3.3	sorted	14
2.3.4	aveknt	14

1 目前已经（部分）实现的函数列表

1.1 样条函数生成

1. `csapi` 三次 pp 格式样条函数生成，默认使用 not-a-knot 边界条件。
2. `csape` 三次 pp 格式样条函数生成，可由用户指定边界条件。
3. `ppmak` 根据用户给出的信息创建若干 pp 格式的样条曲线。
4. `bspline` 根据输入节点创建一个 B-Spline 基函数，并返回其 pp 格式的表达式。
5. `spapi` 根据给定的 B 样条节点或样条曲线次数以及插值条件创建一个 B 格式的样条曲线。
6. `spmak` 根据用户给出的信息创建一个 B 格式的样条曲线。

1.2 样条函数后置处理

1. `fnval` 返回样条在一个点或一组点处的函数值。
2. `fn2fm` 将样条函数在不同格式间转换。目前仅支持一维 B 格式样条函数向一维 pp 格式样条函数的转换。
3. `fnder` 返回样条函数的 d 阶导数，负值时返回特定的 $|d|$ 阶不定积分。
4. `fnbrk` 得到样条函数的相关信息

1.3 断点、节点、位点

1. `aptknt` 根据给出的数据位点以及样条阶数生成合适的 B 样条基函数节点序列
2. `augknt` 根据所需要的样条阶数扩充输入的节点序列
3. `sorted` 查找一组点在另一组非递减序列中的相对位置索引。
4. `aveknt` 根据给出的节点序列生成一个新的节点序列。

2 函数功能说明与使用样例

注意：目前函数只支持一维情形下的一元样条插值和拟合，高维和多元函数暂不支持。

2.1 样条函数生成

2.1.1 `csapi`

使用 `csapi` 函数时请至少输入四个插值点，更少插值点情形的功能待完善。

`csapi` 函数根据给定的插值点列使用非扭结边界条件 (not-a-knot)，共有 2 种用法：

I. `pp = csapi(x,y)`，该函数返回值为一个结构体 (pp 格式)，其中

- `x` `y` 分别为样条插值坐标点的横纵坐标构成的一维向量，长度必须相同，且要求 `x` 已经完成单增排列；
- `pp` 为一结构体，包含如下字段：`form` (样条的形式，在这里为 pp 格式)、`breaks` (样条节点的位置，为一维向量)、`coefs` (分片多项式系数组成的矩阵，每行为一个三次多项式的系数)、`pieces` (样条的多项式片数)、`order` (分片多项式的阶数)、`dim` (目标函数的维数)；

如输入：

```
x = [0 1 2 3];
y = [1 2 4 6];
s = csapi(x,y)
```

输出为:

```
form : 'pp'
breaks: [0 1 2 3]
order : 4
coefs : [3 * 4 double]
pieces: 3
dim : 1
```

II. `values = csapi(x,y,xx)`, 该函数返回生成样条函数在 `xx` 处的值, 其中

- `x y` 分别为样条插值坐标点的横纵坐标构成的一维向量, 长度必须相同, 且要求 `x` 已经完成单增排列;
- `xx` 为待求值点横坐标构成的一维向量;
- `value` 为一维向量, 返回待求值点的函数值;

如输入:

```
x = [0 1 2 3];
y = [0 1 4 9];
xx = [0.5 1.5];
values = csapi(x,y,xx)
```

输出为:

```
values = [0.7500 2.7500]
```

2.1.2 csape

使用 `csape` 函数函数时请至少输入四个插值点, 更少插值点情形的功能待完善。

`csape` 函数根据给定的插值点列和边界条件信息生成对应的三次样条函数 (pp 格式), 其返回值为一个结构体, 包含生成的三次样条函数的节点, 分片多项式系数等信息。(下文中左侧指在样条函数的第一个节点处, 右侧指在最后一个节点处)

`csape` 函数的用法为 `S = csape(x, [e1 y e2], conds)`, 其中

- `x y` 分别为样条插值坐标点的横纵坐标构成的向量 (建议将 `x` 进行单增排列后作为参数输入);
- `e1 e2` 分别为样条插值在左右两侧的边界条件值 (根据 `conds` 的取值将会被理解为不同的含义, 甚至被忽略);
- `conds` 指定样条插值的边界条件, 其值可以为字符串和 1×2 int 类型矩阵。

I. 当不指定 `conds` 时, 即使用 `S = csape(x, y)` 时, 函数将默认在两侧使用 not-a-knot 边界条件, 函数行为与 `csapi` 相同。

II. 当 `conds` 为字符串时, 其对应的取值和意义如下:

"complete" or "clamped"	指定样条函数左右两侧的一阶导数分别为 e1 e2， 若未提供 e1 e2 的输入，将会以“默认值”进行计算。
"not-a-knot"	样条函数在第二个节点和倒数第二个节点三次连续， 此边界条件会忽略输入的 e1 e2。
"second"	指定样条函数左右两侧的二阶导数分别为 e1 e2， 若未提供 e1 e2 的输入，将会默认 e1 = e2 = 0， 此情形下函数行为与边界条件为 "variational"时相同。
"periodic"	指定样条函数左右两侧的一阶导数和二阶导数均相等， 此边界条件会忽略输入的 e1 e2。
"variational"	指定样条函数左右两侧的二阶导数均为 0， 此边界条件会忽略输入的 e1 e2。

其他字符串内容（大小写敏感）会被认定为非法输入。“默认值”说明详见下文。

III. 当 `conds` 为 1×2 `int` 类型矩阵时，矩阵元素只能取值于 0, 1, 2，当出现其他取值的矩阵元素或未指定指的矩阵元素时，函数会将对应的元素认定为 1 并以“默认值”进行计算。例如：

```
S = csape(x, [e1 y e2], [-1 2]) 等价于 S = csape(x, [default y e2], [1 2]);
S = csape(x, [e1 y e2], [2 , ]) 等价于 S = csape(x, [e1 y default], [2 , 1])
```

0, 1, 2 取值分别对应于 "periodic", "complete", "second"，意为指定两侧对应阶数的导数值，例如：

```
S = csape(x, [e1 y e2], [1 2]) 将会指定三次样条函数 S 左侧一阶导数为 e1，右侧二阶导数为 e2；
S = csape(x, [e1 y e2], [2 1]) 将会指定三次样条函数 S 左侧二阶导数为 e1，右侧一阶导数为 e2；
```

"periodic"边界条件即矩阵元素 0 不能与其他条件混用，当 `conds` 矩阵中出现 0 元素时，必须两个元素均为 0，否则为 0 的元素将被视为 1 并以“默认值”进行计算。例如：

```
S = csape(x, [e1 y e2], [0 0]) 等价于 S = csape(x, [e1 y e2], "periodic");
S = csape(x, [e1 y e2], [0 1]) 等价于 S = csape(x, [default y e2], [1 1]);
S = csape(x, [e1 y e2], [2 0]) 等价于 S = csape(x, [e1 y default], [2 1]);
```

关于“默认值”的说明：当函数输入参数错误或者不足时，函数会自动启动默认值计算。以左侧为例，默认计算指将样条函数左侧的一阶导数值设定为前四个插值点唯一确定的三次多项式在第一个节点处的一阶导数值，右侧同理。

（其实这里将矩阵元素 0 对应周期边界条件是不合适的：首先，因为已经可以通过字符串的形式设定周期边界条件，再将 0 对应周期边界条件是一种重复。其次，矩阵形式的边界条件指定方式本意是取混合边界条件，即两侧边界条件不同，但是周期边界条件本身不能与其他边界条件混用，仍将其用矩阵元素表示是和数学理论的不对应，也会带来编程实现上的不自然。最后，因为将 0 对应到了周期边界条件，not-a-knot 边界条件将无法同其他边界条件混合使用，若要使用 not-a-knot 边界条件，将只能两端同时为之，这是函数功能上的一种缺失。将矩阵元素 0 对应到 not-a-knot 边界条件是最自然清晰的选择，但是这里软件开发一个要求是与 MATLAB 兼容，这是不得不做出的权衡和让步。）

2.1.3 ppmak

ppmak 函数根据用户给出的信息创建若干 pp 格式的分段多项式，目前提供 3 种用法：

I. `pp = ppmak(breaks,coefs)`，该函数返回值为一个结构体（pp 格式），其中

- `breaks` 为节点横坐标构成的一维向量，要求其已完成单增排列；
- `coefs` 为二维矩阵，由分片多项式的系数构成。

– 列数 `l = (length(breaks) - 1)*Order`，`Order` 为分段多项式的次数，由此可见我们要求输入的节点必须满足 `length(breaks)-1` 能够整除 `coefs` 矩阵的列数；

- 行数决定了分段多项式的数量或者说维数，即矩阵的每一行是一个以 `breaks` 为节点，以 `coefs` 中对应行为系数的分段多项式。
- 更加确切的说，若令 k 表示分段多项式的次数，则 `coefs(:,i*k+j)` 即为该行表示的分段多项式的第 $(i+1)$ 个分片多项式的第 j 个系数，这里的系数是按从高次到低次的顺序排列的。
- `pp` 为一结构体，包含如下字段：
 - `form` (样条的形式，在这里为 `pp` 格式)
 - `breaks` (样条节点的位置，为一维向量，与输入的 `breaks` 相同)
 - `coefs` (分段多项式系数组成的矩阵，与输入的 `coefs` 矩阵有所区别，接下来会详细介绍)
 - `pieces` (每一维分段多项式的多项式片数，即 `length(breaks) - 1`)
 - `order` (分段多项式的阶数)
 - `dim` (目标函数的维数，即输入的 `coefs` 的行数)。
 - 在这里需要特别注意的是，输出结果中的 `coefs` 矩阵会重新排序，与输入的 `coefs` 矩阵不同。其列数为分段多项式的次数 `Order`，其行数 $r = (\text{length}(\text{breaks})-1)*d$ ， d 为分段多项式的维数。具体来说 `coefs(i*d+j,:)`，其中 $j = 1, 2, \dots, d$ 为第 j 维分段多项式的第 $(i+1)$ 片多项式的系数。

如输入：

```
p1 = ppmak([1 3 4],[1 2 5 6;3 4 7 8])
```

输出为：

```
form : 'pp'
breaks: [1 3 4]
coefs : [4 * 2 double]
pieces: 2
order : 2
dim : 2
```

输出中的 `coefs` 矩阵为：

```
[1 2;3 4;5 6;7 8];
```

从中可以看出返回的结构体中的系数矩阵与输入的 `coefs` 矩阵的区别。

II. `[pp1,pp2,...,ppm] = ppmak(breaks,coefs)`，该函数与上面的函数类似，但其返回值为多个结构体 (`pp` 格式)，每个结构体的维数都是 1 维，也即将第一种用法中的输出结果拆分为多个 1 维的情形。其输入参数与第一种用法相同，且要求输出参数的个数必须等于 `coefs` 矩阵的行数。

III. `pp = ppmak(breaks,coefs,d)`，该函数与第一种用法类似，返回值为一个结构体 (`pp` 格式)，但其输入参数个数和方式与第一种用法有所不同，其中

- `breaks` 为节点横坐标构成的一维向量，要求其已完成单增排列；
- `coefs` 为二维矩阵，由分片多项式的系数构成，其形式与上面第一种用法中的不同。在此用法中，要求系数矩阵的形式与第一种用法中输出结果中的系数矩阵的形式相同，具体形式可参见函数的第一种用法。于是在该用法中，输入的 `coefs` 矩阵与输出结果中的 `coefs` 矩阵完全相同。
- `d` 为一标量，表示分段多项式的维数；
- `pp` 为一结构体，具体可见该函数第一种用法中该结构体的介绍，需要注意的是这里结构体中的 `coefs` 矩阵没有调整顺序，与输入的 `coefs` 矩阵相同。

如输入：

```
p2 = ppmak([1 3 4],[1 2;3 4;5 6;7 8],2)
```

输出为：

```
form : 'pp'
breaks: [1 3 4]
coefs : [4 * 2 double]
pieces: 2
order : 2
dim : 2
```

其结果与第一种用法中的完全相同，只不过该种用法提供了内部使用中的系数矩阵的排列。

2.1.4 bspline

bspline 根据输入节点创建一个 B-Spline 基函数，并返回其 pp 格式的表达式。

I. $p = \text{bspline}(t)$ ，该函数返回值为一个结构体，表示根据输入节点序列 t 生成的 B-Spline 基函数的 pp 格式。节点序列 t 要求满足单增排列，且无重复元素。（注：1. B-Spline 是样条函数除 pp 格式以外的另一种表达方式，其以 B-Spline 基函数为基础，但若表示某个基函数，仍需使用 pp 格式；2. 当节点序列 t 中有重复元素时，重复元素对应的节点成为重节点，暂不支持此种用法。）

如输入：

```
t = [1, 2, 3, 4, 5];
p = bspline(t)
```

输出为：

```
form : 'pp'
breaks : [1 2 3 4 5]
coefs : [4×4 double]
pieces : 4
order : 4
dim : 1
```

II. $\text{bspline}(t)$ ，绘制根据节点序列 t 生成的 B-Spline 基函数和该基函数的所有分片多项式。注意此种用法不能有变量接受函数的返回值。

如输入：

```
t = [1, 2, 3, 4, 5];
bspline(t)
```

输出为一个图像，包含生成的 B-Spline 基函数和该基函数的所有分片多项式。

（注：这种用法可能需要在函数内部调用 `plot` 等函数，若在代码实现中绘图函数调用比较困难，需询问北太天元工作人员，实现完成后请删除本注。）

2.1.5 spapi

根据给定的 B 样条节点或样条曲线阶数以及插值条件创建一个 B 格式的样条曲线。

- $s = \text{spapi}(\text{knots}, x, y)$ 创建一个样条曲线。
 - knots 为 B 样条的节点。
 - x 为插值条件节点，服从单增序列要求，可以重复。
 - y 为上述插值条件节点上的值，如有重复，重复的值被分别视为在该点处的 $1, 2, \dots$ 阶导数值。
 - x, y 必须为长度相同的一维向量。
 - 返回的 s 的阶数 k 满足 $k = \text{length}(\text{knots}) - \text{length}(x)$ 。
- $s = \text{spapi}(k, x, y)$ 创建一个 k 阶样条曲线。

- x, y 为插值条件，要求与 $s = \text{spapi}(\text{knots}, x, y)$ 相同。
- 返回的 s 是满足插值条件与指定 B 样条节点或指定阶数的样条曲线，为 B 格式。

I. 在 B 样条节点 $0, 0, 0, 0, 1, 2, 2, 2, 2$ 上进行 B 样条插值，插值条件为 $s(0) = 2, s(1) = 0, s'(1) = 1, s''(1) = 2, s(2) = -1$ 。

即输入：

```
knots = [0, 0, 0, 0, 1, 2, 2, 2, 2];
x = [0, 1, 1, 1, 2];
y = [2, 0, 1, 2, -1];
s = spapi(knots, x, y);
```

输出为：

```
form : 'B-'
knots : [0 0 0 0 1 2 2 2 2]
coefs : [2 -0.3333 -0.3333 1.0000 -1]
number : 5
order : 4
dim : 1
```

II. 在 $[0, 4]$ 上对函数 $f(x) = \sin(x)$ 进行 5 次 B 样条拟合，采样点为区间上均匀分布的 41 个点 $0, 0.1, 0.2, \dots, 3.9, 4$ 。

即输入：

```
x = 0:0.1:4;
s = spapi(6, x, sin(x));
```

输出为：

```
form : 'B-'
knots : [0 0 0 0 0 0 0.3000 0.4000 0.5000 0.6000 0.7000 0.8000 ... ]
coefs : [0 0.0600 0.1400 0.2390 0.3543 0.4806 0.5661 0.6458 ... ]
number : 41
order : 6
dim : 1
```

实现说明

- $s = \text{spapi}(\text{knots}, x, y)$
 - 相容性检查（尚未完全实现）
 - 样条插值。只需要对 x 前后各扩充 $k-1$ 个节点，由于目标样条函数可以被线性组合 $\sum \alpha_j B_j$ 的方式表达，而 B_j 以及重节点情形下 $B_j^{(n)}$ 都是可以被显式表达的，问题转化为求解系数 α_j 。其中，以 α_j 为自变量的这一系列方程组的系数矩阵的各行表示各 j 情形下 B_j 或者其导数在各节点处的取值，右端项为对应的已知值 y_j ，它们是稀疏的，Matlab 的官方文档给出的求解方式是「调用 `spcol` 来提供几乎块对角线搭配矩阵 $(B_{j,k}(x))$ ，`slvblk` 使用块 QR 分解求解线性系统」。
- $s = \text{spapi}(k, x, y)$
 - 实现 `knots = augknt(knots, k)`: 将 `knots(begin)` 和 `knots(end)` 各重复 k 遍，例如 `augknt([1,2,3],2)`，输出 `[1,1,2,3,3]`。只需读取参数 Vector，对将返回的临时 Vector 遍历赋值时，在 `[0]` 和 `[v.size()+k-2]` 处使用 `for` 循环填入需重复的数据即可。
 - 实现 `knots = aveknt(knots, k)`: 将 `knots` 去掉头尾元素后，对每 $k-1$ 个元素取平均数构成新的 `knots`。只需嵌套双层循环，外层遍历参数 Vector 的 `[1]` 至 `[v.size()-k+1]`，内层循环累加这 $k-1$ 个数并取平均值，赋值于将要返回的临时 Vector 即可。

- 实现 `knots = aptknt(tau, k)`: 即调用 `augknt([tau(1), aveknt(tau,k), tau(end)], k)`.
- 调用 `s = spapi(aptknt(x, k), x, y)`, 即可得到结果。

2.1.6 spmak

`spmak` 函数根据用户给出的信息创建一个 B 格式的样条曲线。其用法如下:

`sp = spmak(knots,coefs)`, 该函数返回值为一个结构体 (B 格式), 其中

- `knots` 为 B 样条节点, 这里要求其为单增非降的一维向量;
- `coefs` 为 B 样条的系数, 要求其一维向量, 且长度必须小于 `knots` 的长度;
- 样条曲线的阶数 `k` 满足 `k = length(knots) - length(coefs)`
 - 需要注意, 节点的重数需要满足 $\leq k$ 。如果对应的 B-spline 只有一个不同的节点, 即 `knots(j) = knots(j+k)`, 那么系数 `coefs(:,j)` 将被直接忽略。
- `sp` 为一结构体, 包含如下字段: `form` (样条的形式, 在这里为 'B-' 格式)、`knots` (样条节点的位置, 为一维向量)、`coefs` (B 样条基函数系数构成的一维数向量)、`number` (B 样条基函数的个数)、`order` (样条函数的阶数)、`dim` (目标函数的维数);

如给定 B 样条节点 `{0,0,1,2,3,4,5,5}`, 可以得到三个 4 次的 B 样条, 若这三个 B 样条的系数分别为 1, 4 和 -2, 则我们可以得到对应的 B 格式样条曲线。

输入为:

```
knots = [0 0 1 2 3 4 5 5];  
coefs = [1 4 -2];  
sp = spmak(knots,coefs)
```

输出为:

```
coefs : [1,4,-2]  
dim : 1  
form : 'B-'  
knots : [0,0,1,2,3,4,5,5]  
number : 3  
order : 5
```


2.2 样条函数后置处理

2.2.1 fnval

fnval 函数返回样条在一个点或一组点处的函数值。

- $y = \text{fnval}(s, x)$ 返回样条函数 s 在 x 处的值, x 可以为标量、一维向量, x 不能在 s 的定义域之外。返回值 y 的值取决于 x 。
- $\text{fnval}(s, x)$ 同 $\text{fnval}(x, s)$ 。

2.2.2 fn2fm

- $ns = \text{fn2fm}(s, \text{form})$ 将样条函数 s 转换为 form 格式, 返回转换后的样条函数 ns 。
- form 由字符向量或字符串标量形式指定, 形式的选择分别为 'B-'、'pp', 分别为 B 格式、pp 格式。
- B-form 将函数描述为给定结序列的给定阶 k 的 B 样条的加权和, pp 格式用其局部多项式系数来描述一个函数。

如输入:

```
knots = [0,0,0,0,1,2,2,2,2];  
x = [0,1,1,1,2];  
y = [2,0,1,2,-1];  
s = spapi(knots, x, y);  
ns = fn2fm(s, 'pp');
```

输出为:

```
form: 'pp'  
breaks: [0 1 2]  
coefs: [2 * 4 double]  
pieces: 2  
order: 4  
dim: 1
```

2.2.3 fnder

fnder 计算一个样条函数的导函数, 并仍以样条函数的形式返回。

- $ds = \text{fnder}(s, \text{order})$ 得到样条函数 s 的 order 阶导函数, s 可以是 pp 格式、B 格式。
- order 为正数代表对原函数求导数, order 为负数代表对原函数求积分 (积分常数设为 0)。
- order 是一个不超过样条函数次数的正整数。
- 该函数输出的形式与输入相同, 仍以样条函数的格式返回, 结果要么是 pp 格式, 要么是 B 格式。
- $\text{fnder}(s)$ 同 $\text{fnder}(s, 1)$ 。

如输入:

```
x = [ 0, 2, 3, 5, 6 ];  
y = [1, 0, 3, 1, -1, 0, 0];  
s = csape(x,y, 'complete');  
ds = fnder(s);
```

输出为:

```
form: 'pp'
breaks: [0 2 3 5 6]
coefs: [4 * 3 double]
pieces: 4
order: 3
dim: 1
```

B 格式下 fnder 函数实现说明: B 格式下 fnder 函数实现分为两个部分：求导和求积分。

记输入的节点序列为 $[t_0, t_1, \dots, t_m, t_{m+1}]$ 的 B 格式样条函数为

$$S(x) = \sum_{i=1}^{m-n+1} c_i B_i^n(x) \quad (1)$$

根据 B 样条基函数的求导规则（[这里要求 \$n \geq 2\$ ，暂时如此实现](#)）

$$\frac{d}{dx} B_i^n(x) = \frac{n}{t_{i+n-1} - t_{i-1}} B_i^{n-1}(x) - \frac{n}{t_{i+n} - t_i} B_{i+1}^{n-1}(x) := \alpha_i B_i^{n-1}(x) - \beta_i B_{i+1}^{n-1}(x) \quad (2)$$

对 $S(x)$ 进行求导可得

$$\begin{aligned} S'(x) &= \sum_{i=1}^{m-n+1} c_i [B_i^n(x)]' \\ &= \sum_{i=1}^{m-n+1} c_i [\alpha_i B_i^{n-1}(x) - \beta_i B_{i+1}^{n-1}(x)] \\ &= c_1 \alpha_1 B_1^{n-1}(x) + \sum_{i=2}^{m-n+1} (c_i \alpha_i - c_{i-1} \beta_{i-1}) B_i^{n-1}(x) - c_{m-n+1} \beta_{m-n+1} B_{m-n+2}^{n-1}(x) \end{aligned} \quad (3)$$

根据以上推导可以实现 B 格式下 fnder 函数求导数的部分。（需要注意的一点是，因为重节点的存在，以上公式中 $B_i^{n-1}(x), B_{m-n+2}^{n-1}(x)$ 的结点可能是全部相同的，根据基函数的定义，这样的 B 样条基函数就是 0 函数，MATLAB 的做法是在 $S'(x)$ 的计算结果中将这样的基函数剔除，于是会出现求导过后反而构成 $S(x)$ 的基函数个数变少的情况。）

对于求积分问题，设

$$S(x) = \sum_{i=1}^{\text{Number}} y_i B_i^n,$$

通过在右侧添加重节点的方式，将 S 的基函数进行延拓，求 S 的原函数等价于求

$$S + 0 * \sum_{i=1}^{\text{Order} + 1 - \text{multinum}} B_{\text{Number} + i}^n$$

的原函数。其中 multinum 为最右侧重节点数，当最后一个点非重节点时 multinum = 1，若重节点过多，则报错。

接下来考虑 S 的原函数，在上述得到的节点最后再添加一个重节点。则可以以

$$\{B_i^{n+1}\}_1^{\text{Number} + \text{Order} + 1 - \text{multinum}}$$

为基函数进行待定系数，再根据 B-spline 求导规则式 (2)，可以列出线性方程组。关键在于此时

$$\frac{d}{dx} B_{last}^{n+1} = \frac{n+1}{t_{last+n+1-1} - t_{last-1}} B_{last}^n + 0.$$

该方程组为：

$$\begin{pmatrix} \alpha_1 & & & & \\ -\alpha_2 & \alpha_2 & & & \\ & -\alpha_3 & \ddots & & \\ & & \ddots & \alpha_{N-1} & \\ & & & \alpha_N & \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{N-1} \\ y_N \end{pmatrix}.$$

2.2.4 fnbrk

fnbrk 用来得到样条函数的相关信息
I.fnbrk 的用法 1 为 [out1,...,outn] = fnbrk(f,part1,...,partm), 要求 $n \leq m$

- part_i 表示第 i 个输入的样条信息，详见下文
- f 表示输入的样条函数
- out_i 输出第 i 个输入的样条信息

对于任意格式的样条曲线 f，part_i 可以是如下形式：

"form"	f 的格式, 返回'ppform' 或'B-form'
"variables"	f 的自变量的维数, 返回一个整型标量
"dimension"	f 的维数, 返回一个整型标量
"coefficients"	f 的系数, 对 pp 格式, 返回一个矩阵, 其列数是样条函数的阶数, 行数 $r = (\text{length}(\text{breaks}) - 1)$, 每行是一个多项式系数的降幂排列。对 B 格式, 返回一个一维向量, 其每个分量表示对应的 B 样条的系数
"interval"	f 的有效区间, 返回一个 1×2 的 double 型矩阵
"order"	f 的阶数, 返回一个整型标量

对于 pp 格式的样条曲线 f，part_i 还可以是如下形式：

"breaks"	f 的插值节点, 返回一个一维向量
"pieces"	f 的多项式段数, 返回一个整型标量

对于 B 格式的样条曲线 f，part_i 还可以是如下形式：

"knots"	f 的 B 样条节点, 返回一个一维向量
"number"	f 的系数的数量, 返回一个整型标量

I. 输入：

```
x = [ 0,2,3, 5,6 ];
y = [1,0,3,1,-1,0,0];
f = csape(x,y,'complete');
[form,variables,dimension,coefs,interval,order,breaks,pieces]...
=fnbrk(f,'form','variables','dimension','coefficients','interval','order','breaks','pieces')
```

输出为：

```
form = 'ppform'
variables = 1
dimension = 1
coefs =
    -0.6499    1.5497    1.0000         0
         0.9489   -2.3495   -0.5995    3.0000
         0.1142    0.4973   -2.4516    1.0000
        -1.0914    1.1828    0.9086   -1.0000
interval = 0    6
order = 4
```

```
breaks = 0    2    3    5    6
pieces = 4
```

II. 输入:

```
knots = [0,0,0,0,1,2,2,2,2];
x = [0,1,1,1,2];
y = [2,0,1,2,-1];
f = spapi(knots,x,y);
[form,variables,dimension,coefs,interval,order,knots,number] = ...
fnbrk(f,'form','variables','dimension','coefficients','interval','order','knots','number')
```

输出为:

```
form = 'B-form'
variables = 1
dimension = 1
coefs =
    2.0000   -0.3333   -0.3333    1.0000   -1.0000
interval = 0    2
order = 4
knots = 0    0    0    0    1    2    2    2    2
number = 5
```

II. fnbrk 的用法 2 为 out=fnbrk(f,interval)，可以与用法 1 混用，即 f 之后的参数类型既可以是矩阵，也可以是字符串

- interval 为一个大小为 1 × 2 的矩阵，表示区间
- f 表示输入的样条函数
- out 输出 f 截取在区间 interval 的 pp 格式样条结构体

2.3 断点、节点、位点

2.3.1 aptknt

aptknt 根据给出的数据位点以及样条阶数生成合适的 B 样条基函数节点序列
用法为 knots = aptknt(tau, k) 或 [knots, k] = aptknt(tau, k)

- tau 为一个单增的序列，并满足 $\forall i, \tau(i) < \tau(i + k - 1)$
- k 为生成的样条阶数， $k \geq 2$
- knots 为生成的节点序列
- 返回值 k 为可选项，为实际能够实现的样条阶数，因为所给数据位点不一定能够满足实现 k 阶样条的条件

生成算法为：首尾节点重复 k 次，中间节点采用 $\tau^*(i) = \frac{\sum_{j=i+1}^{i+k-1} \tau(j)}{k-1}$ 计算
即 aptknt(tau, k) 等价于 augknt([tau(1),aveknt(tau,k),tau(end)],k)

样例 I.
输入:

```
knots = aptknt( [1 2 3 4 5 6], 3 )
```

输出为:

```
knots =
    1.0000    1.0000    1.0000    2.5000    3.5000    4.5000    6.0000    6.0000    6.0000
```

样例 II.

输入：

```
[knots, k] = aptknt( [1 2 3 4 5 6], 7 )
```

输出为：

knots =

```
1 1 1 1 1 1 1 6 6 6 6 6 6 6
```

k =

```
1x1 int32
```

```
6
```

2.3.2 augknt

augknt 扩充输入的节点序列

输入的节点无要求，函数会先将节点序列排序去重，下面的描述中的首尾节点以及内节点都指排序去重后的节点序列，共两种用法 `augknt(knots,k)` 或 `augknt(knots,k,mults)`

- knots 为输入的节点序列
- k 边界节点的重数
- multi 内节点的重数，为正整数或者正整数数组

I. `augknt(knots,k)` 根据给出的 knots 返回一个单增的扩充节点序列满足只在首尾两个节点有重复，重复 k 次（所以节点序列可能会变短）；以及一个整数表示左端节点增加的重数（因此可以为负数），可以缺省

样例：

输入：

```
[knots, addl] = aptknt( [1 2 3 3 3], 2 )
```

输出为：

knots =

```
1 1 2 3 3
```

addl =

```
1x1 int32
```

```
1
```

% 结果与 `[knots, addl] = aptknt( [3 2 3 1 3], 2 )` 相同

II. `augknt(knots,k, mults)` 边界点同 `augknt(knots,k)`，内节点会重复 multi 次，并且若输入的 multi 为数组并且长度与内节点数量相同那么第 j 个内节点将会重复 multi(j) 次，否则都重复 multi(1) 次（描述中的下标从 1 开始记）。同样有 addl 的返回值，同 `augknt(knots,k)`，样例中不再演示

样例 1：

```
knots = augknt( [4 3 2 3 1 3 5], 4, 2 )
```

输出为：

```
augknot =
```

```
1 1 1 1 2 2 3 3 4 4 5 5 5 5
```

样例 2:

```
knots = augknt( [4 3 2 3 1 3 5], 4, [2 3] )
```

输出为:

```
augknot =
```

```
1 1 1 1 2 2 3 3 4 4 5 5 5 5
```

样例 3:

```
knots = augknt( [4 3 2 3 1 3 5], 4, [2 3 1] )
```

输出为:

```
augknot =
```

```
1 1 1 1 2 2 3 3 3 4 5 5 5 5
```

2.3.3 sorted

`sorted` 函数用于查找一组点在另一组非递减序列中的相对位置索引。

- `pointer = sorted(meshsites, sites)` 返回一个整数行向量 `pointer`，其中每个条目 `j` 等于 `sites` 的单增序列的第 `j` 项在 `meshsites` 中小于或等于 `sites(i)` 的条目数量。
- 此函数特别适用于在拟合或插值过程中定位数据点相对于基准点集（如结点序列）的位置。

如输入:

```
meshsites = [1 2 3 3 4 5];
sites = [2 3 1 4 3];
pointer = sorted(meshsites, sites);
```

输出为:

```
pointer: [1 2 4 4 5]
```

此输出表示 `sites` 中的每个点相对于 `meshsites` 的位置。例如，第一个 1 表示 `sites` 中最小的元素 1 的位置在 `meshsites` 中的相对位置索引。

2.3.4 aveknt

`aveknt` 用于计算节点数组元素的平均值。具体来说，它根据给定的节点序列 `t` 和样条曲线的阶数 `k`，返回连续 `k - 1` 个节点的平均值作为新的节点位置。

用法为: `tstar = aveknt( t, k )` 注意: 在 `k = t` 的长度的时候，返回的新节点序列长度为 `tatsr` 为空向量。

- `t` 为一个单调递增的节点序列。
- `k` 为样条曲线的阶数， $k \geq 2$ 且 `k` 为整数。
- `tstar` 为生成的新节点序列。

生成算法为: $t_i^* := \frac{t_{i+1} + \dots + t_{i+k-1}}{k-1}$, $i = 1:n$, 其中 `n` 为节点序列 `t` 的长度, t_i^* 为新的节点序列。输入:

```
x = [1,2,3,3,3];
k = 3;
tstar = aveknt(x, k);
```

输出:

```
tstar =
```

```
2.5000    3.0000
```

输入：

```
y = [1,2,3];
```

```
k = 3;
```

```
tstar = aveknt(y, k);
```

输出：

```
tstar =
```

```
1x0 empty double
```